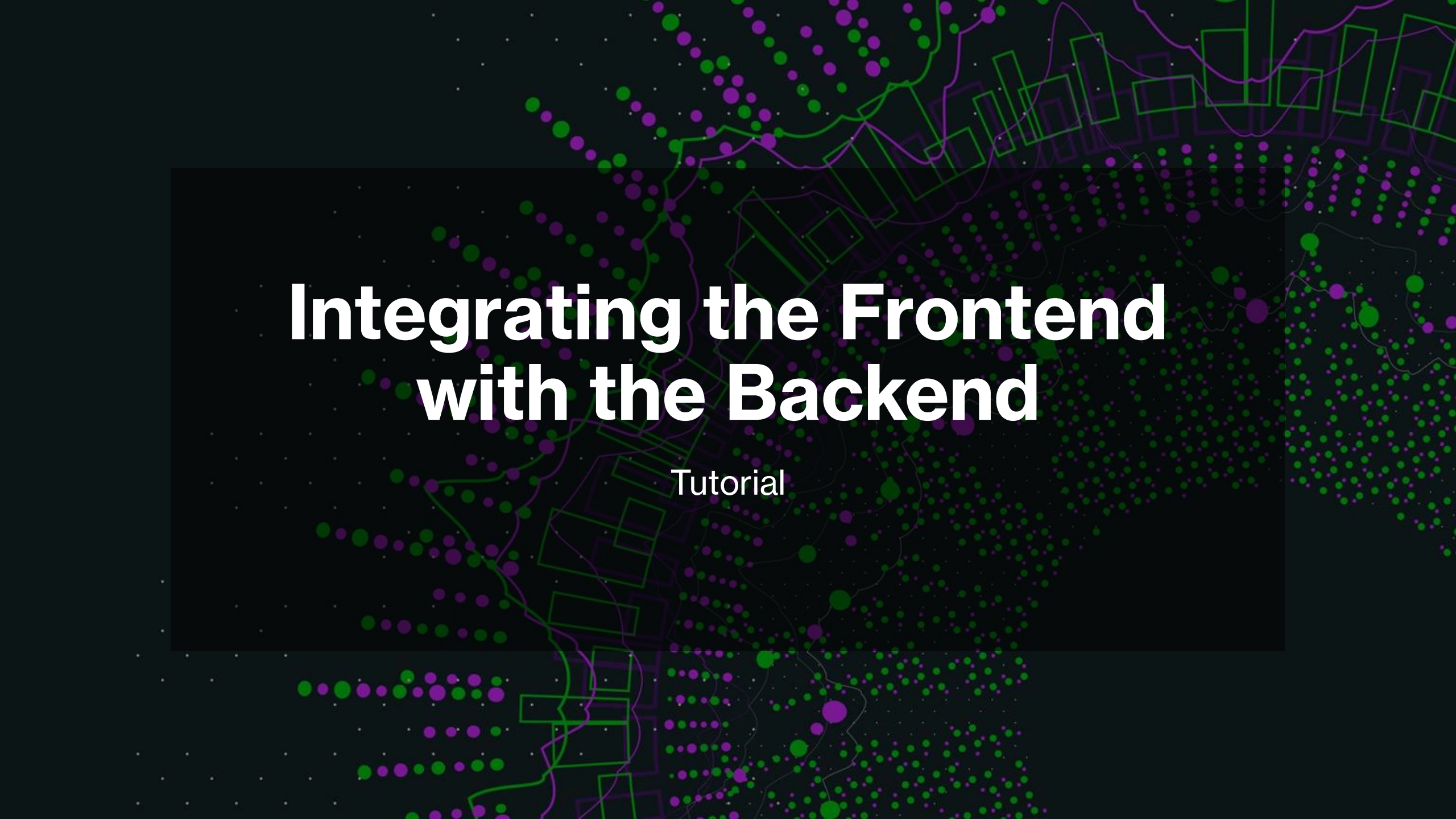


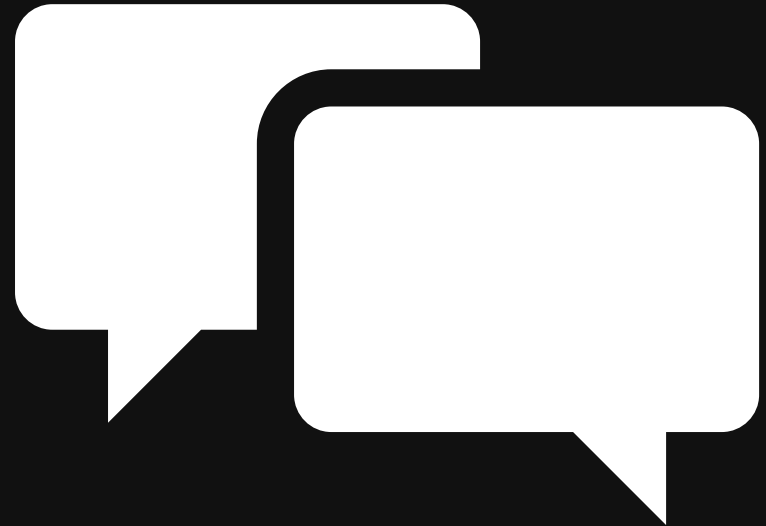


Integrating the Frontend with the Backend

Tutorial

Overview

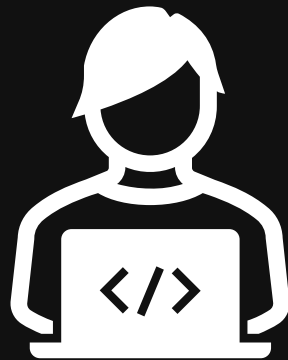
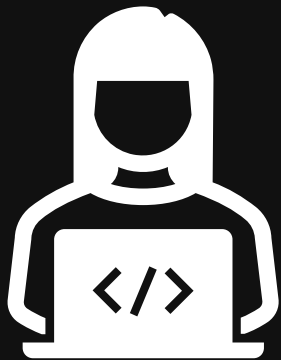
- Creating an API
- Calling the API endpoint from frontend
- Running the application
- Live Demo



Creating an API

- Set up a backend server
- Define routes or endpoints
- Receive and handle client requests
- Return responses in a structured format
 - normally JSON

Creating an API



```
app > backend > app.py > ...
1  from backend.calculator import calculate_multiple
2  from flask import Flask, request, jsonify
3  from flask_cors import CORS
4
5  app = Flask(__name__)
6  CORS(app, origins=["http://localhost:5173"])
7
8  @app.route('/calculate', methods=['POST'])
9  def calculate_route():
10     data = request.json
11     expression = data.get("expression")
12
13     try:
14         res = calculate_multiple(expression)
15     except ValueError as e:
16         return jsonify({"error": str(e)}), 400
17     except Exception as e:
18         return jsonify({"error": "Server error"}), 500
19
20     return jsonify({"result": res}), 200
21
22 if __name__ == '__main__':
23     app.run(debug=True)
```

app > backend > app.py > ...

```
1 from backend.calculator import calculate_multiple
2 from flask import Flask, request, jsonify
3 from flask_cors import CORS
```

```
4
5 app = Flask(__name__)
6 CORS(app, origins=["http://localhost:5173"])
```

```
7
8 @app.route('/calculate', methods=['POST'])
```

```
9 def calculate_route():
10     data = request.json
11     expression = data.get("expression")
12
13     try:
14         res = calculate_multiple(expression)
15     except ValueError as e:
16         return jsonify({"error": str(e)}), 400
17     except Exception as e:
18         return jsonify({"error": "Server error"}), 500
19
20     return jsonify({"result": res}), 200
```

```
21
22 if __name__ == '__main__':
23     app.run(debug=True)
```

```
1 from flask import Flask, jsonify
2 from flask_cors import CORS
3
4 app = Flask(__name__)
5 CORS(app, origins=["http://localhost:5173"])
6
7 @app.route('/calculate', methods=['POST'])
8 def calculate_route():
9     data = request.json
10     expression = data.get("expression")
11
12     try:
13         res = calculate_multiple(expression)
14     except ValueError as e:
15         return jsonify({"error": str(e)}), 400
16     except Exception as e:
17         return jsonify({"error": "Server error"}), 500
18
19     return jsonify({"result": res}), 200
20
21 if __name__ == '__main__':
22     app.run(debug=True)
```

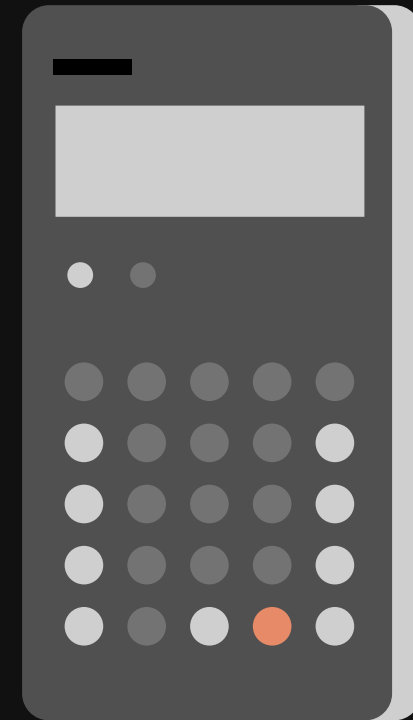
Calling the API

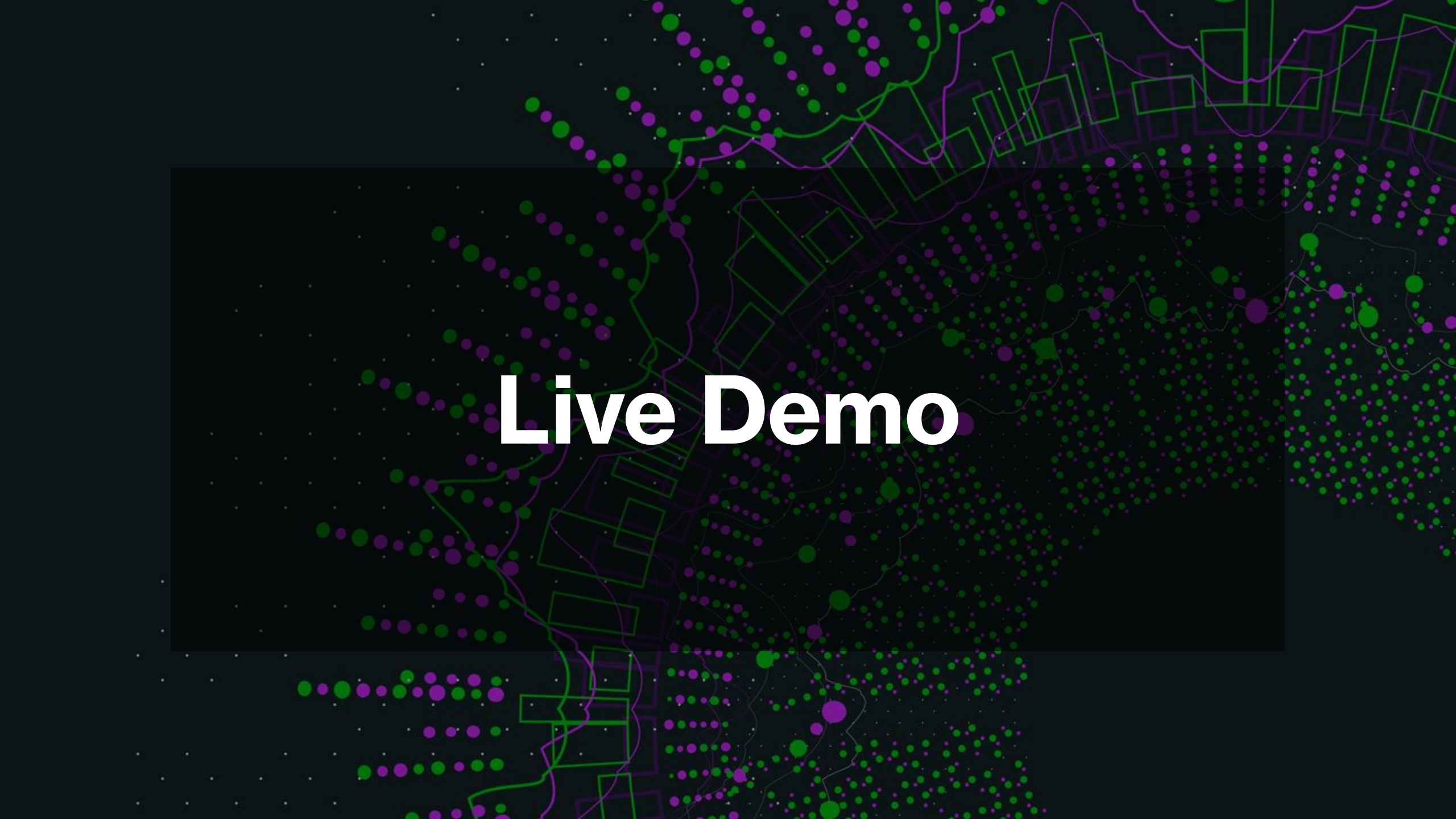
- Create an asynchronous frontend function
- Prepare and validate input
- Use `fetch()` to call the required endpoint in the async function
- Process the JSON response returned from the backend

```
84  ✓ const calculateOutput = async () => {
85      let calculatorValidInput = calculatorInput.value.replaceAll('÷', '/')
86      calculatorValidInput = calculatorValidInput.replaceAll('x', '*')
87
88      const isValidInput = validateInput(calculatorValidInput)
89      ✓ if (!isValidInput) {
90          calculatorOutput.value = "Error";
91          return;
92      }
93
94  ✓   const res = await fetch('http://localhost:5000/calculate', {
95      method: 'POST',
96  ✓   headers: {
97       'Content-Type': 'application/json'
98   },
99   body: JSON.stringify({expression: calculatorValidInput})
100  });
101
102   const data = await res.json();
103
104  ✓   if (data.error)
105   {
106       alert(data.error);
107       calculatorOutput.value = "Error";
108   }
109  ✓   else
110   {
111       const isValidOutput = validateOutput(data.result)
112  ✓   if (!isValidOutput) {
113       calculatorOutput.value = "Error";
114       return;
115   }
116       calculatorOutput.value = data.result;
117   }
118 }
```

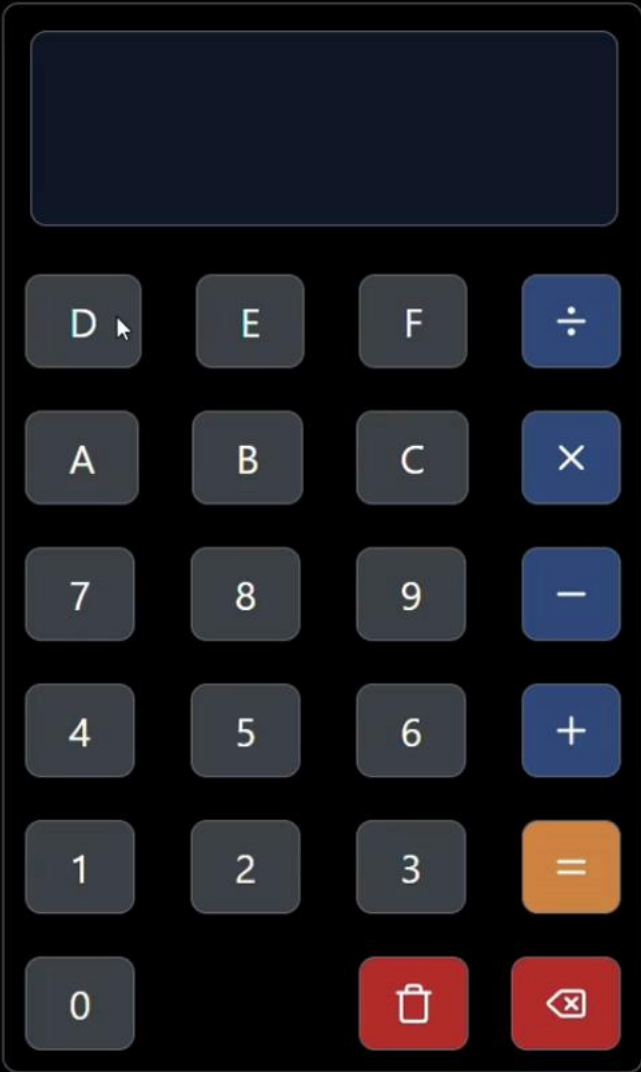
Running the Application

- Start the backend server
- Start the frontend development server
- Open the frontend URL in a browser



The background is a dark gray field filled with a complex pattern of green and purple dots, lines, and rectangles. The dots are arranged in various clusters and sequences, some appearing as straight lines and others as more chaotic groups. Thin, wavy lines in green and purple meander across the space, often passing through or around the clusters of dots. Rectangles, also in green and purple, are scattered throughout, some appearing as simple outlines and others as more complex, overlapping shapes. The overall effect is a dense, abstract, and somewhat organic-looking composition.

Live Demo





Next Step

Unit Tests for Frontend